Movie Awards Voting Management System

Project submitted to the

School of Engineering & Technology, BML Munjal University
for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Technology

Submitted by

Kinshuk Mishra (240881)

Anurag Singh (240390)

Arav Jain (240394)

Under the Guidance of

Dr. Nishtha Phutela

BML Munjal University
Kapriwas, Haryana– 122413

[May, 2026]

Certificate

Date: 05/05/2026

This is to certify that the work present in this Project entitled "**Movie Awards Voting Management System**" has been carried out by **Kinshuk Mishra, Anurag Singh** and **Arav Jain** under my supervision. The work is genuine, original, and suitable for submission to the BML Munjal University for the award of Bachelor of Technology in **School of Engineering and Technology**.

Supervisor

Dr. Nishtha Phutela

Designation: Associate Professor

Table of Contents

Sl.No.	Topic	Page No.
1	Certificate	2
2	Table of Contents	3
3	Abstract	4
4	Abbreviations	4
5	Introduction	5
6	Methodology	6
7	Observations	13
8	Concluding Remarks	14
9	Societal Relevance	14
10	Future Scope	14
11	Team Details and Contribution	14
12	References	14

Abstract

This project focuses on the development of a centralized Database Management System (DBMS) for a Movie Awards Voting System to efficiently manage movie nominations, categories, user roles, and voting processes. The system is designed using a relational database to ensure data integrity, reduce redundancy, and maintain consistency across all entities such as users, movies, nominations, and votes.

The system is implemented using MySQL as the backend database with structured SQL queries, views, and stored procedures to handle complex operations such as vote casting, result generation, and nomination management. Transactions are used to ensure atomicity and consistency during critical operations like voting and nomination updates.

The effectiveness of the system is demonstrated through optimized query execution using joins, aggregate functions, and indexing, which improves data retrieval performance for reporting and result calculation. The proposed system ensures a transparent, efficient, and reliable voting mechanism suitable for award management scenarios.

Abbreviations

Short Form	**Full Form**
DBMS	Database Management System
SQL	Structured Query Language
ER	Entity-Relationship

Introduction

As data-driven systems become increasingly important in modern applications, efficient database management plays a crucial role in handling large volumes of structured information. In domains such as entertainment, governance, and organizational decision-making, it is essential to maintain accurate records of participants, categories, and voting results in a consistent and reliable manner.

This project focuses on the design and implementation of a Movie Awards Voting System using a Relational Database Management System (RDBMS). The system is developed to manage and streamline the processes of nomination creation, category assignment, jury voting, and result computation.

The database is designed using normalization principles to reduce redundancy and ensure data integrity. Key DBMS concepts such as primary and foreign keys, joins, views, stored procedures, and transactions are implemented to support efficient data handling and maintain consistency across operations.

Although this system is implemented for movie award management, the same design principles can be applied to other real-world voting systems such as elections, talent competitions, corporate decision-making processes, and online polling platforms. These systems similarly require secure, transparent, and consistent handling of votes and results.

Overall, the system provides a structured and scalable solution for managing voting-based applications, ensuring transparency in results and efficient data processing through optimized SQL queries.

Methodology

Design (ER Diagram, Schema, Normalization)

- **ER Diagram:**

  The ER diagram represents the relationships between key entities such as Users, Movies, People, Categories, Award_Category, Nominations, and Votes. It defines how users participate in voting, how movies are linked to people (actors, directors, etc.), and how nominations are assigned to specific award categories.

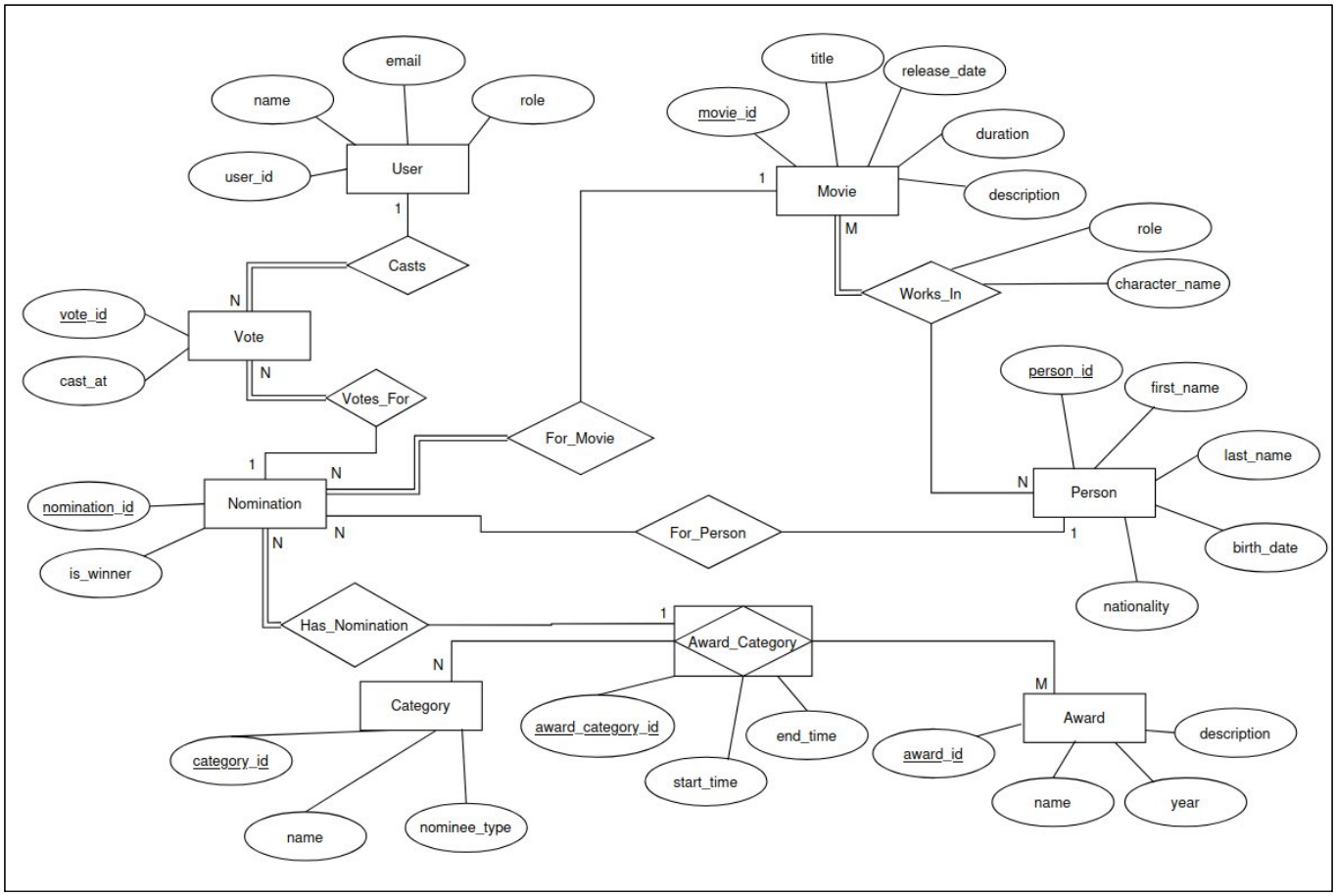

Fig 1. ER Diagram

- **Schema:**

  The relational schema consists of multiple tables with clearly defined primary and foreign keys to maintain data integrity.

1. **Users-** User authentication and roles such as ADMIN and JURY.
   Primary Key: user_id
2. **Movies-** Stores movie details such as title, release year, duration, and description.
   Primary Key: movie_id
3. **People-** Stores information about actors, directors, and other crew members.
   Primary Key: person_id
4. **Movie_Crew-** Represents the relationship (Works_In) between Movies and People, specifying the role of each person in a movie such as ACTOR, DIRECTOR, etc., along with optional character names.
   Primary Key: {movie_id, person_id}
   Foreign Keys: movie_id, person_id
5. **Awards-** Stores information about award events, including name and year.
   Primary Key: id
6. **Categories-** Defines award categories and their nominee types, such as MOVIE, ACTOR, DIRECTOR, etc.
   Primary Key: id
7. **Award_Category**- Links Awards with Categories and defines the voting time window for each category within an award.
   Primary Key: id
   Foreign Keys: award_id, category_id
8. **Nominations-** Maps movies and/or people to specific award categories, representing nominees.
   Primary Key: id
   Foreign Keys: award_category_id, movie_id, person_id
9. **Votes-** Stores jury votes, with constraints ensuring that each user can vote only once per category.
   Primary Key: id
   Foreign Keys: jury_id, award_category_id, nomination_id

Primary keys uniquely identify records (e.g., id), while foreign keys establish relationships between tables, ensuring referential integrity and consistency across the database.

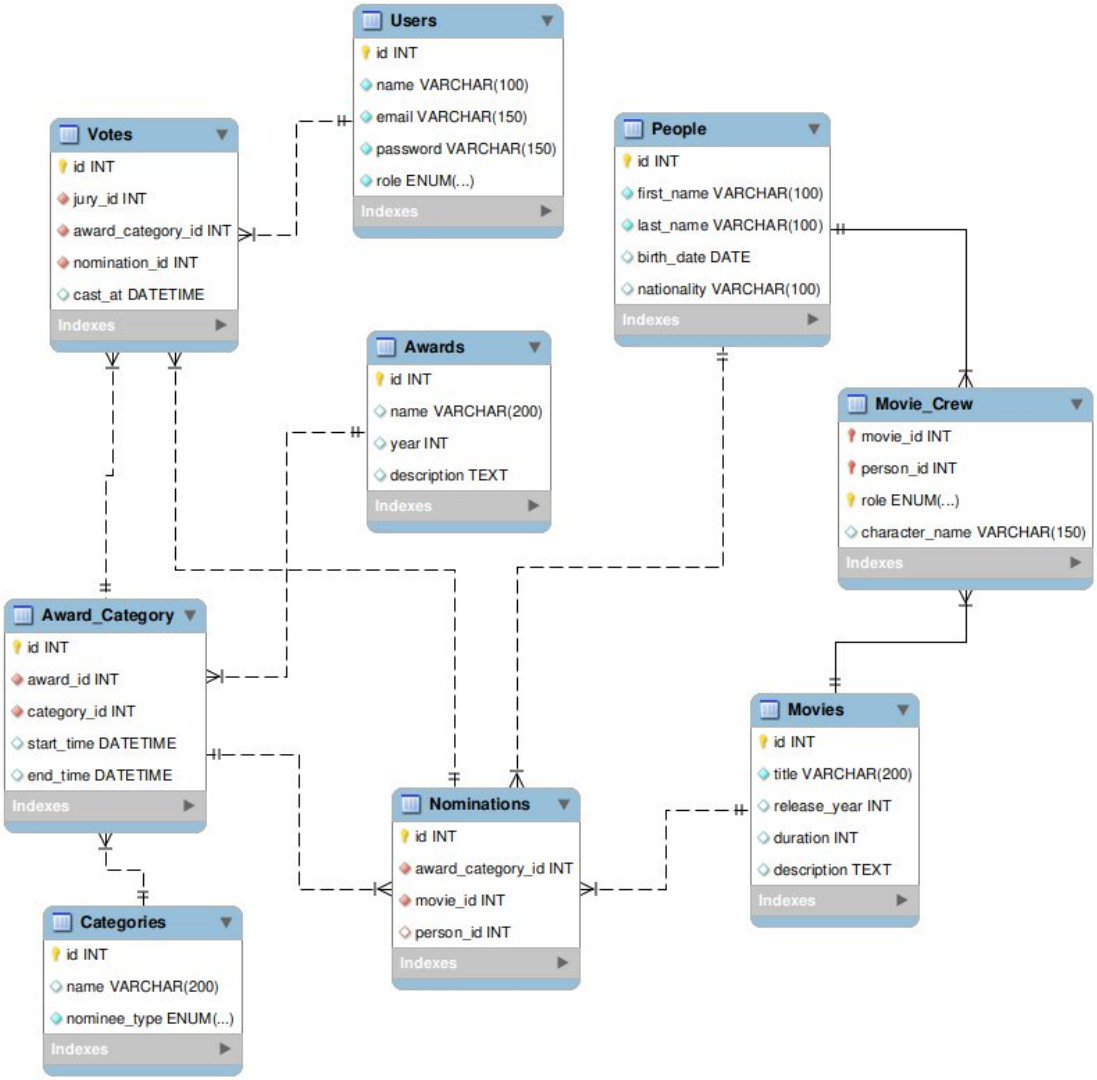
Users
id INT
name VARCHAR(100)
email VARCHAR(150)
password VARCHAR(150)
role ENUM(...)
Indexes
Votes
id INT
jury_id INT
award_category_id INT
nomination_id INT
cast_at DATETIME
Indexes
People
id INT
first_name VARCHAR(100)
last_name VARCHAR(100)
birth_date DATE
nationality VARCHAR(100)
Indexes
Awards
id INT
name VARCHAR(200)
year INT
description TEXT
Indexes
Movie_Crew
movie_id INT
person_id INT
role ENUM(...)
character_name VARCHAR(150)
Indexes
Award_Category
id INT
award_id INT
category_id INT
start_time DATETIME
end_time DATETIME
Indexes
Movies
id INT
title VARCHAR(200)
release_year INT
duration INT
description TEXT
Indexes
Nominations
id INT
award_category_id INT
movie_id INT
person_id INT
Indexes
Categories
id INT
name VARCHAR(200)
nominee_type ENUM(...)
Indexes

Fig 2. Schema

- **Normalization:** The database is normalized up to 3NF (Third Normal Form) to eliminate redundancy and ensure data consistency.

  1NF: Ensures atomic values in all tables

  2NF: Removes partial dependencies by separating entities like Movies and People

  3NF: Removes transitive dependencies by separating Categories, Awards, and Votes into independent tables

  This design minimizes redundancy and prevents insertion, update, and deletion anomalies.

  Votes Table does not follow 3NF:
  Structure**:**
  Votes(id, jury_id, award_category_id, nomination_id, cast_at)

  Functional Dependencies:
  id → jury_id, award_category_id, nomination_id, cast_at
  nomination_id → award_category_id

  This violates 3NF as a transitive dependency exists:
  id → nomination_id → award_category_id

  award_category_id depends on nomination_id instead of directly on the primary key.

  Reason for Design Choice:
  The attribute award_category_id is intentionally included to enforce the constraint: UNIQUE (jury_id, award_category_id) to improve performance.

Implementation

The system is implemented using a React.js (Vite) front-end and a MySQL relational database as the backend. The backend logic is handled through structured SQL queries, views, and stored procedures to ensure data consistency, integrity, and efficient querying.

It supports operations such as nomination management, vote casting, result computation, and statistical analysis.

1. User Authentication

Input: Email, Password
Output: Authorized access

Steps:

1. Validate credentials from Users table.
2. Identify role (ADMIN/JURY).
3. Grant access based on role.

2. Nomination Process

Input: Award Category, Movie, Person (optional)
Output: Nomination entry

Steps:

1. Validate award category exists.
2. Validate nominee type (movie/person).
3. Insert into Nominations.
4. Enforce uniqueness using index (award_category_id, movie_id, person_id).

3. Voting Process

Input: Jury ID, Nomination ID
Output: Vote recorded

Steps:

1. Validate user role = JURY.
2. Fetch award_category_id from nomination.
3. Check voting time window (start_time ≤ current_time ≤ end_time).
4. Ensure jury has not already voted in that category.
5. Insert vote into Votes table.

4. Result Calculation

Input: Award Category
Output: Winner

Steps:

1. Count votes per nomination.
2. Group by nomination_id.
3. Select nomination with maximum votes.
4. Return winner details.

Views Used

Vote_Count

- Calculates total votes per nomination
- Used for result computation

Winners

- Identifies highest voted nomination per category
- Used for final result display

Stored Procedures & Transactions

1. Voting Transaction

Purpose: Ensure a vote is recorded correctly

Steps:

1. Start transaction
2. Lock nomination row and fetch its category
3. Lock existing vote for the same jury and category
4. Check duplicate vote condition
5. Insert vote
6. Commit transaction

Failure Handling:

- If any step fails then rollback
- Prevents invalid votes

2. Add Nomination Transaction

Purpose: Ensure valid nomination entry

Steps:

1. Start transaction
2. Lock category row
3. Fetch nominee type
4. If category = MOVIE then insert nomination with NULL person
5. Else:
   Validate person belongs to movie crew based on role
   Insert nomination
6. Commit transaction

Failure Handling:

- Rollback if constraint violation occurs

3. Update Nomination Transaction

Purpose: Ensure consistency when updating nomination and related votes

Steps:

1. Start transaction
2. Lock nomination row
3. Fetch and store previous category
4. Update nomination details
5. Lock all related votes
6. Update votes to reflect new category
7. Commit transaction

Failure Handling:

- If any step fails then rollback
- Prevents mismatch between nomination and votes
- Ensures referential consistency

Indexes Used

A unique index is created on the columns (award_category_id, movie_id, COALESCE(person_id, 0)) in the Nominations table to prevent duplicate nominations. An index on (award_category_id) in Nominations is used for faster filtering by category. In the Votes table, indexes are created on (nomination_id) and (award_category_id) to optimize vote counting and category-based queries. Additionally, an index on (movie_id, person_id) in the Movie_Crew table is used to speed up validation and join operations.

Observations

Aspect	Observation
Normalization	Data is well-structured across multiple entities. For example, movie details are stored in Movies, while people are stored separately in People, allowing the same person to be linked to multiple movies without duplication via Movie_Crew.
Denormalization	In the Votes table, `award_category_id` is stored along with `nomination_id` to enforce the rule that a jury can vote only once per category using a UNIQUE constraint.
Data Integrity	Referential integrity is maintained using foreign keys. For example, a nomination must reference a valid movie and category, and a vote must reference an existing nomination.
Indexing	Indexes improve performance in multiple scenarios: fetching all nominations of a category (Nominations), counting votes for results (Votes), and validating whether a person is part of a movie crew (Movie_Crew).
Transactions	Transactions are used in operations involving multiple dependent steps. For example, updating a nomination also updates all related votes to maintain consistency across tables.
Locking	Row-level locking ensures consistency during concurrent operations. For example, while creating a nomination, the category row is locked to prevent changes during validation, and during voting, existing votes are locked to avoid duplicate entries.
Stored Procedures	Core operations are encapsulated in procedures. For example, CreateNomination validates nominee type (movie vs crew), CastVote enforces voting rules, and UpdateNomination ensures related data stays consistent.
Joins Usage	Joins are used extensively for combining data. For example, displaying nominations requires joining Movies, People, and Nominations, while category statistics involve joining Categories, Award_Category, Nominations, and Votes.
Consistency	Data consistency is maintained across operations. For example, when a nomination’s category is changed, all associated votes are updated to reflect the new category.

Concluding Remarks

The Movie Awards Management System demonstrates how a well-structured DBMS can efficiently manage complex relationships between movies, people, nominations, and voting processes. It ensures data integrity, reduces redundancy, and supports concurrent operations through transactions and constraints, resulting in a reliable and scalable system.

Societal Relevance

This system simplifies and digitizes award management processes, making them more transparent and efficient. It reduces manual errors in nomination and voting processes and ensures fair evaluation through controlled voting mechanisms. Such systems can be applied to film industries, academic awards, or any large-scale evaluation platforms.

Future Scope

Future improvements of the system can focus on strengthening database-level features such as audit logging to track important operations like voting and nomination changes, and advanced reporting to generate insights like voting trends and category-wise statistics using optimized queries. The system can also be enhanced with better concurrency control mechanisms to handle high-volume simultaneous voting more efficiently. Additionally, improvements in scalability through indexing optimization or partitioning, along with proper backup and recovery mechanisms, can further increase reliability and ensure data safety in case of failures.

Team Details and Individual Contribution

- **Kinshuk Mishra:** Contributed to frontend development of the website, assisted in video preparation, and actively participated in ER diagram design and overall database (DBMS) planning.
- **Arav Jain:** Worked on backend development, led schema and database design including ER diagram preparation, contributed to video creation, and supported overall DBMS implementation and structure.
- **Anurag Singh:** Prepared the PPT, assisted in video creation, and contributed to ER diagram design and database planning as part of the DBMS project along with coordination support.

References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts*.
2. Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of Database Systems*.

AI Usage

AI tools were used to assist in understanding database design concepts, refining SQL queries, developing website front-end and improving documentation structure. All core design decisions, schema creation, and implementation logic were developed independently.